SIMATS ENGINEERING

SWETHA.B

192412215

EXPERIMENT 4

ITA0606

MACHINE LEARNING

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

df = pd.read_csv("/content/play_tennis (1).csv")

print("================ DATASET ================\n")
display(df)

print("\nDataset Shape:", df.shape)
print("\nColumn Names:")
print(df.columns.tolist())

print("\n================ MISSING VALUES ================\n")
print(df.isnull().sum())

df = df.dropna()

X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

```
print("\n================ INPUT FEATURES ================\n")
display(X)

print("\n================ TARGET ================\n")
print(y)

X = pd.get_dummies(X)

print("\n================ ENCODED FEATURES ================\n")
display(X)

encoder = LabelEncoder()
y = encoder.fit_transform(y)

print("\n================ ENCODED TARGET ================\n")
print(y)

print("\nTarget Encoding:")
for i, label in enumerate(encoder.classes_):
    print(label, "=", i)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\n================ DATA SPLIT ================\n")
print("Training samples:", len(X_train))
```

```
print("Testing samples :", len(X_test))

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=2000,
    random_state=42
)
model.fit(X_train, y_train)

print("\n================ TRAINING ================\n")
print("ANN training completed successfully!")

y_pred = model.predict(X_test)

print("\n================ PREDICTION ================\n")
print("Actual values    :", y_test)
print("Predicted values :", y_pred)

# Convert back to Yes/No
actual = encoder.inverse_transform(y_test)
predicted = encoder.inverse_transform(y_pred)

print("\nActual classes    :", actual)
```

```
print("Predicted classes :", predicted)

accuracy = accuracy_score(y_test, y_pred)

print("\n================ ACCURACY ================\n")
print("Accuracy:", accuracy)
print("Accuracy (%):", accuracy * 100)

print("\n================ CLASSIFICATION REPORT ================\n")

print(classification_report(
    y_test,
    y_pred,
    target_names=encoder.classes_,
    zero_division=0
))

cm = confusion_matrix(y_test, y_pred)

print("\n================ CONFUSION MATRIX ================\n")
print(cm)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=encoder.classes_
).plot()

plt.title("ANN Confusion Matrix")
plt.show()
```

```
plt.figure(figsize=(7,4))

plt.plot(model.loss_curve_)

plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("ANN Training Loss using Backpropagation")
plt.grid(True)
plt.show()
print("\n================================================")
print("FINAL RESULT")
print("================================================")
print("Algorithm : Artificial Neural Network")
print("Training  : Backpropagation")
print("Dataset   : Play Tennis")
print("Accuracy  :", round(accuracy * 100, 2), "%")
print("================================================")
```

OUTPUT:

```
================ DATASET ================
```

	day	outlook	temp	humidity	wind	play
0	D1	Sunny	Hot	High	Weak	No
1	D2	Sunny	Hot	High	Strong	No
2	D3	Overcast	Hot	High	Weak	Yes
3	D4	Rain	Mild	High	Weak	Yes
4	D5	Rain	Cool	Normal	Weak	Yes
5	D6	Rain	Cool	Normal	Strong	No
6	D7	Overcast	Cool	Normal	Strong	Yes
7	D8	Sunny	Mild	High	Weak	No
8	D9	Sunny	Cool	Normal	Weak	Yes
9	D10	Rain	Mild	Normal	Weak	Yes
10	D11	Sunny	Mild	Normal	Strong	Yes
11	D12	Overcast	Mild	High	Strong	Yes
12	D13	Overcast	Hot	Normal	Weak	Yes
13	D14	Rain	Mild	High	Strong	No

```
Dataset Shape: (14, 6)

Column Names:
['day', 'outlook', 'temp', 'humidity', 'wind', 'play']
```

```
================ MISSING VALUES ================

day         0
outlook     0
temp        0
humidity    0
wind        0
play        0
dtype: int64
```

```
================ INPUT FEATURES ================
```

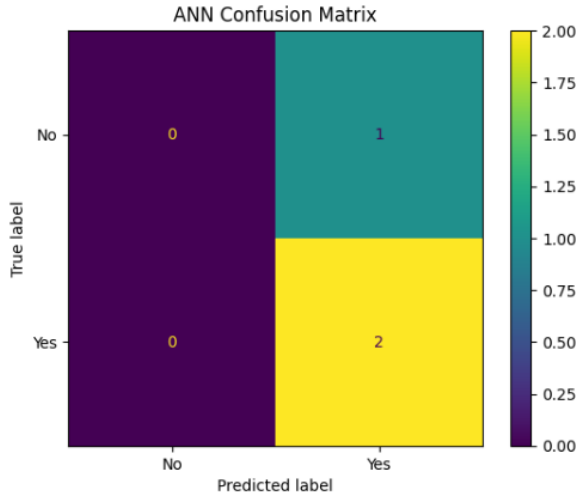
	day	outlook	temp	humidity	wind
0	D1	Sunny	Hot	High	Weak
1	D2	Sunny	Hot	High	Strong
2	D3	Overcast	Hot	High	Weak
3	D4	Rain	Mild	High	Weak
4	D5	Rain	Cool	Normal	Weak
5	D6	Rain	Cool	Normal	Strong
6	D7	Overcast	Cool	Normal	Strong
7	D8	Sunny	Mild	High	Weak
8	D9	Sunny	Cool	Normal	Weak
9	D10	Rain	Mild	Normal	Weak
10	D11	Sunny	Mild	Normal	Strong
11	D12	Overcast	Mild	High	Strong
12	D13	Overcast	Hot	Normal	Weak
13	D14	Rain	Mild	High	Strong

```
================ TARGET ================

0      No
1      No
2     Yes
3     Yes
4     Yes
5      No
6     Yes
7      No
8     Yes
9     Yes
10    Yes
11    Yes
12    Yes
13     No
Name: play, dtype: object

================ ENCODED FEATURES ================
```

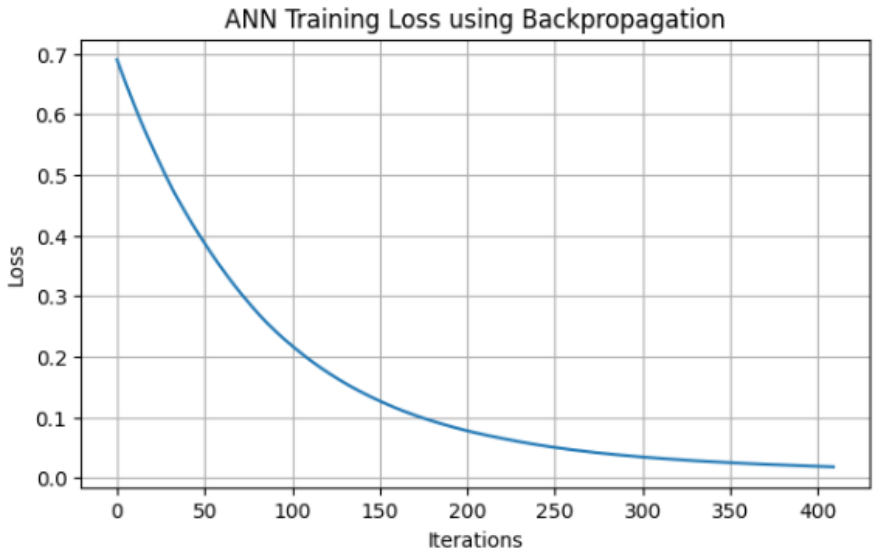
	day_D1	day_D10	day_D11	day_D12	day_D13	day_D14	day_D2	day_D3	day_D4	day_D5	...	outlook_Overcast	outlook_Rain	outlook_Sunny	temp_Cool	temp_Hot	temp_Mild	humidity_High	humidity_Normal	wind_Strong	wind_Weak
0	True	False	False	False	False	False	False	False	False	False	...	False	False	True	False	True	False	True	False	False	True
1	False	False	False	False	False	False	True	False	False	False	...	False	False	True	False	True	False	True	False	True	False
2	False	False	False	False	False	False	False	True	False	False	...	True	False	False	False	True	False	True	False	False	True
3	False	False	False	False	False	False	False	False	True	False	...	False	True	False	False	False	True	True	False	False	True
4	False	False	False	False	False	False	False	False	False	True	...	False	True	False	True	False	False	False	True	False	True
5	False	False	False	False	False	False	False	False	False	False	...	False	True	False	True	False	False	False	True	True	False
6	False	False	False	False	False	False	False	False	False	False	...	True	False	False	True	False	False	False	True	True	False
7	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	False	True	True	False	False	True
8	False	False	False	False	False	False	False	False	False	False	...	False	False	True	True	False	False	False	True	False	True
9	False	True	False	False	False	False	False	False	False	False	...	False	True	False	False	False	True	False	True	False	True
10	False	False	True	False	False	False	False	False	False	False	...	False	False	True	False	False	True	False	True	True	False
11	False	False	False	True	False	False	False	False	False	False	...	True	False	False	False	False	True	True	False	True	False
12	False	False	False	False	True	False	False	False	False	False	...	True	False	False	False	True	False	False	True	False	True
13	False	False	False	False	False	True	False	False	False	False	...	False	True	False	False	False	True	True	False	True	False

14 rows × 24 columns

```
================ ENCODED TARGET ================
[0 0 1 1 1 0 1 0 1 1 1 1 1 0]

Target Encoding:
No = 0
Yes = 1

================ DATA SPLIT ================

Training samples: 11
Testing samples : 3

================ TRAINING ================

ANN training completed successfully!

================ PREDICTION ================

Actual values    : [1 1 0]
Predicted values : [1 1 1]

Actual classes    : ['Yes' 'Yes' 'No']
Predicted classes : ['Yes' 'Yes' 'Yes']

================ ACCURACY ================

Accuracy: 0.6666666666666666
Accuracy (%): 66.66666666666666

================ CLASSIFICATION REPORT ================

              precision    recall  f1-score   support

          No       0.00      0.00      0.00         1
         Yes       0.67      1.00      0.80         2

    accuracy                           0.67         3
   macro avg       0.33      0.50      0.40         3
weighted avg       0.44      0.67      0.53         3


================ CONFUSION MATRIX ================

[[0 1]
 [0 2]]
```



Predicted label




```
================================================
FINAL RESULT
================================================
Algorithm : Artificial Neural Network
Training  : Backpropagation
Dataset   : Play Tennis
Accuracy  : 66.67 %
================================================
```